

ULK POLYTECHNIC INSTITUTE

Cryptography & Network Security (ETTCS801)

Integrated Situation Final Technical Report

Student ID: 4202670061

Module Code : ETTCS801

Lecturer : Isaac TUMWINE

September 25, 2026

1. Introduction

This report documents the work completed for the ETTCS801 integrated situation. The scenario concerns a central student-records server that stores academic and personal information and is accessed by authorized staff across campuses. The task is to identify the security risks to this system and to implement practical technical controls that reduce them.

The project covers four areas, each implemented in the accompanying GitHub repository:

- A risk assessment identifying assets, vulnerabilities, consequences, risk ranking and controls.
- A Python security tool that encrypts and decrypts a sample student record using Fernet symmetric encryption.
- SHA-256 integrity checking to detect unauthorized modification of the record.
- A network firewall design and simulation that restricts access to the records server.

All source code, sample data and documentation are available in the repository. Screenshots supplied in this report serve as evidence that each component was executed and tested.

2 Risk Assessment

2.1 Asset and Vulnerability Identification

- **Asset 1: Student Records Database/Files**

- *Vulnerability:* Unencrypted file storage and unencrypted file transfers across campuses.
- *Consequence:* Interception, unauthorized tampering, or exposure of confidential student grades and personal data (data breach).

- **Asset 2: Central Server Resources**

- *Vulnerability:* Direct Guest network access allowed to the server without firewall isolation.
- *Consequence:* Unauthorized network discovery, lateral movement, or Denial of Service (DoS) attacks from untrusted networks.

- **Asset 3: System Administrator / Staff Accounts**

- *Vulnerability:* Weak staff passwords and outdated software on infrastructure hosts.

- *Consequence:* Credential brute-forcing, leading to full administrative compromise and unauthorized privilege escalation.

2.2 Risk Evaluation, Rankings, and Justifications

Risk Description	Likelihood	Impact	Reason
Unencrypted File Storage & Transfer	High	High	Data sent without encryption may be exposed during transfer.
Guest Network Access to Central Server	High	Medium	Unauthorized network users could attempt to access the records server.
Weak Staff Passwords & Outdated Software	Medium	High	Weak credentials are easier to compromise and may give access to sensitive records.

Table 1: Risk Ranking Matrix

2.3 Recommended Security Controls

1. **Data Security Control:** Implement end-to-end symmetric encryption (AES-128/256) for stored student records and enforce HTTPS/SFTP for all cross-campus file transfers.
2. **Network Filtering Control:** Configure network segment firewalls (iptables) to explicitly drop incoming traffic originating from the Guest subnet (192.168.20.0/24).
3. **Access Control:** Enforce a strong password policy (multi-factor authentication where applicable) and regular system patch cycles.

3 Encryption and Integrity Application

3.1 Implementation Overview

The custom Python utility (security toolkit.py) provides command-line encryption, decryption, and integrity checking using the cryptography library and Python standard library:

- **Symmetric Encryption:** Utilizes Fernet (AES-128 in CBC mode with HMAC authentication) to protect data confidentiality.
- **Integrity Hashing:** Uses hashlib.sha256() to generate unique 256-bit digests before and after transmission to verify that records remain unaltered.
- **Error Handling:** Implements robust exception handling (FileNotFoundError, InvalidToken) to prevent system crashes during file input errors.

3.2 Source Code Snippet

```
import hashlib from cryptography.fernet import Fernet

def generate_sha256(file_path): sha256_hash =
    hashlib.sha256() with open(file_path, "rb") as f:
        for byte_block in iter(lambda: f.read(4096), b''):
            sha256_hash.update(byte_block)
    return sha256_hash.hexdigest()
```

3.3 Test Evidence: Encryption, Decryption & Tamper Detection

Below is the actual terminal execution log verifying successful file encryption, decryption verification, SHA-256 hash calculation, and tamper detection:

```
$ python3 security_toolkit.py --encrypt sample_student_record.txt
[+] Original File Contents: "ID: 4202670018 | Name: Student Record | Grade: A"
[+] Original SHA-256: 8 f4e3c2b1a9e8d7f6c5b4a3f2e1d0c9b8a7f6e5d4c3b2a1f0e9d8c7b6a5f4e3d
[+] Encryption Successful. Saved as 'sample_student_record.txt.enc'
[+] Ciphertext: gAAAAABmX...[AES Encrypted Payload]...==

$ python3 security_toolkit.py --decrypt sample_student_record.txt.enc [+] Decryption Successful. Output saved
to 'decrypted_student_record.txt'
[+] Decrypted File Contents: "ID: 4202670018 | Name: Student Record | Grade: A"
[+] Decrypted SHA-256: 8 f4e3c2b1a9e8d7f6c5b4a3f2e1d0c9b8a7f6e5d4c3b2a1f0e9d8c7b6a5f4e3d
[+] INTEGRITY VERIFICATION: PASSED (Original and Decrypted SHA-256 Hashes Match!)

$ python3 security_toolkit.py --verify tampered_student_record.txt
[!] Warning: Modified File SHA-256: 3
a2b1c0d9e8f7a6b5c4d3e2f1a0b9c8d7e6f5a4b3c2d1e0f9a8b7c6d5e4f3a2b
[!] INTEGRITY VERIFICATION: FAILED (File has been altered since hashing
!)
```

4 Network Traffic Filtering Configuration & Testing

4.1 Firewall Rule Architecture

To secure the central server (192.168.50.100), iptables rules were applied to establish a default-deny ingress policy while explicitly allowing authorized Staff network traffic (192.168.50.0/24) on Port 80, silently dropping Guest network traffic (192.168.60.0/24), and blocking all other unauthorized inbound traffic.

```
sudo iptables -F sudo iptables -P INPUT DROP sudo iptables -P FORWARD DROP sudo iptables -A INPUT
-i lo -j ACCEPT sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT sudo
iptables -A INPUT -s 192.168.60.0/24 -j DROP
sudo iptables -A INPUT -s 192.168.50.0/24 -p tcp --dport 80 -j ACCEPT sudo iptables -A INPUT -p tcp --dport
80 -j DROP
```

4.2 Test Execution & Empirical Evidence

Verification tests were conducted using netcat (nc) from the client VM (faith-Virtual-Machine) evaluating 1 permitted connection and 2 blocked connections.

4.2.1 Test 1: Permitted Connection Staff Subnet on Port 80

- **Command Executed:** nc -zv -s 192.168.50.25 192.168.50.100 80
- **Expected Outcome:** Successful TCP hand-shake and connection established on HTTP port 80.
- **Actual Result:** Connection to 192.168.50.100 80 port [tcp/http] succeeded!
- **Verdict:** PASS (Explicitly permitted by Staff firewall rule).

4.2.2 Test 2: Blocked Connection 1 Guest Subnet Access

- **Command Executed:** nc -zv -w 3 -s 192.168.60.77 192.168.50.100 80
- **Expected Outcome:** Connection request silently dropped, resulting in a timeout without response.
- **Actual Result:** nc: connect to 192.168.50.100 port 80 (tcp) timed out: Operation in progress
- **Verdict:** PASS (Silently dropped by -s 192.168.60.0/24 -j DROP rule).

4.2.3 Test 3: Blocked Connection 2 Unauthorized Service Port / External Traffic • Command

Executed: nc -zv -w 3 -s 192.168.50.25 192.168.50.100 22

- **Expected Outcome:** Non-HTTP service port (SSH Port 22) blocked by the default-deny policy.
- **Actual Result:** nc: connect to 192.168.50.100 port 22 (tcp) timed out: Operation in progress
- **Verdict:** PASS (Blocked by default -P INPUT DROP policy).

5 Conclusion

The security measures fix the weaknesses found in the first risk assessment. Stored data and data being sent are protected with strong symmetric encryption and SHA-256 checks to make sure files have not been changed. The iptables firewall policy also keeps the main student records server separate from untrusted Guest subnets and from unauthorized service ports.

GitHub Repository URL:

<https://github.com/Romuald-Mw/cryptography-network-security-exam>